

HireMind

Reimagining how humans are evaluated — intelligently, fairly, and in real time.

Background — The Interview Bottleneck

The job interview has not fundamentally changed in decades. A recruiter schedules a call, asks scripted questions, forms a gut impression, and moves on. At scale, this process breaks down completely. Hiring teams at growing organisations conduct hundreds of first-round interviews per month — most of which are repetitive, inconsistent, and poorly documented. Meanwhile, candidates are evaluated subjectively, with outcomes often influenced by irrelevant factors: how confident they sound, whether they use the right jargon, or whether they remind the interviewer of themselves.

Two structural failures have made this crisis urgent:

The Scalability Wall	Recruiting teams cannot keep pace with application volumes. First-round interviews are a bottleneck that consumes recruiter bandwidth without proportional signal quality.
The Consistency Collapse	Human interviewers are inconsistent by nature. The same candidate evaluated by two different interviewers can receive wildly divergent assessments, making data-driven hiring nearly impossible.
The Jargon Trap	Current evaluation systems reward fluency in technical vocabulary over actual competence. A fresher who understands a concept but lacks the terminology is penalised; a confident but shallow senior is rewarded.
The Black-Box Decision	Interview outcomes are rarely explainable. Candidates receive no structured feedback. Organisations accumulate no structured data. The entire process generates almost zero reusable intelligence.

The result: organisations need an AI-powered interview layer that can conduct, evaluate, and explain candidate assessments — consistently, at scale, and without replicating human bias

The Core Shift

What we are moving away from		What we are building toward
Scripted, static question banks	→	Adaptive follow-up questioning driven by candidate responses
Jargon matching as a proxy for competence	→	Concept comprehension scored independently of vocabulary fluency
Inconsistent human-led first rounds	→	Standardised AI-conducted screening with structured output
Black box pass/fail decisions	→	Auditable reasoning chains with per-criterion scoring
One size fits all interview flows	→	Domain-specific interview logic adapted to role and career path
Fraud detection via honour system	→	Real-time behavioural signals flagging external assistance

The Challenge

Build an AI Interviewer Agent that goes beyond a scripted chatbot. Your system must conduct dynamic, context-aware interviews that adapt to the candidate's responses in real time — asking follow-up questions, detecting surface-level answers, probing deeper where warranted, and producing a structured, explainable evaluation report. The agent must be fair to candidates across experience levels and career backgrounds, and must produce outputs that a human hiring manager can actually act on.

Choose Your Battleground

You are not expected to build a complete AI interviewer from scratch. Pick ONE track and go deep. A working, well-explained solution to a single track beats a shallow attempt at all of them.

TRACK A

The Adaptive Questioning Engine

Moving beyond scripted Q&A; into real-time conversational intelligence

Build an AI interviewer that does not just ask predefined questions — it listens to answers, detects depth, and dynamically generates follow-up questions to probe understanding. The agent should recognise when a candidate has answered superficially and push for elaboration, or when a candidate has demonstrated mastery and move forward efficiently.

PRACTICAL 9-HOUR SCOPE

- Design a prompt architecture where the agent is given a job description and a target competency list
- After each candidate response, the agent evaluates depth (surface / partial / deep) and generates a contextual follow-up or proceeds
- At the end of the session, output a structured report: competency-by-competency score with 1-2 sentence reasoning per criterion
- Test with at least 2 different job descriptions to show adaptability across roles

WHERE TO GET STARTED

- OpenAI / Groq API or Claude API for the agent backbone — use a system prompt to define the interviewer persona
- Prompt chain design: question generation → response evaluation → follow-up or advance logic
- Streamlit for a clean chat-style UI — candidates type responses, agent replies in real time

TRACK B

The Domain-Specific Interview Agent

Because a consulting interview and a coding interview are not the same conversation

Different career paths demand fundamentally different interview structures. A consulting interview revolves around case frameworks, structured problem decomposition, and business hypothesis generation. An engineering interview tests systems thinking and debugging logic. Build an AI interviewer that knows the difference — and adjusts its questioning style, depth probes, and evaluation rubric accordingly.

PRACTICAL 9-HOUR SCOPE

- Choose one or two domains (e.g. consulting + software engineering) and design distinct interview flow logic for each
- For consulting: implement case prompt generation, framework evaluation, and structured feedback (hypothesis quality, structure, communication)
- For engineering: implement system design or DSA-style questioning with adaptive follow-up based on approach
- Allow the user to select domain at the start of the session — the agent reconfigures entirely

WHERE TO GET STARTED

- Public consulting case libraries (CaseFirm, PrepLounge) for domain knowledge and case structures
- LeetCode and system design interview guides for technical domain logic
- Design your prompt system with a domain router: "if domain = consulting, load consulting_system_prompt"

TRACK C

The Fraud & Integrity Detection Layer

Detecting external assistance without penalising genuine inexperience

Build a fraud detection sub-agent that monitors candidate responses for signals of external assistance — copy-pasted answers, vocabulary inconsistency, or sudden shifts in response quality. Critically, this system must be calibrated for fairness: a fresher using simple language is not cheating; a senior suddenly switching to perfect textbook definitions mid-session is a red flag. The system must flag — never auto-reject — suspicious responses, with a confidence score and reasoning note for the human recruiter to review.

PRACTICAL 9-HOUR SCOPE

- Implement a response consistency scorer: track vocabulary complexity and answer structure across the session and flag anomalous spikes
- Build an experience-level calibration layer — the candidate declares their level at the start; the fraud threshold adjusts accordingly
- Flag — do not auto-reject — suspicious responses with a confidence score and a one-line reasoning note for the human recruiter
- Demonstrate with two synthetic sessions: one genuine fresher, one senior candidate with mid-session quality inconsistency

WHERE TO GET STARTED

- spaCy or sentence-transformers for vocabulary and semantic consistency analysis
- Embed each response and track cosine similarity to prior responses — sudden semantic divergence is a signal
- Readability metrics (Flesch-Kincaid, syllable count) as proxies for vocabulary sophistication over time
- Generate synthetic interview transcripts with an LLM to test your detection logic before building UI

TRACK D

The Open-Source Interview Agent Enhancer

Don't build from scratch — make something good dramatically better

Several open-source AI interview tools already exist. Rather than starting from zero, identify an existing open-source AI interviewer, fork it, and build a meaningful capability on top of it that it currently lacks — a structured evaluation report, a domain configuration layer, a real-time coaching mode, a bias audit module, or a post-interview feedback generator for the candidate. The enhancement must be demonstrably functional and clearly differentiated from the base project.

PRACTICAL 9-HOUR SCOPE

- Identify and fork an open-source AI interview project (see Resources below)
- Document what the base project does and does not do — be specific
- Build one clearly scoped enhancement: e.g. adding a post-interview PDF report, a candidate-facing feedback summary, or a bias check
- In your writeup, clearly delineate original code vs your contribution

WHERE TO GET STARTED

- Search GitHub: "AI interviewer", "interview bot", "mock interview agent" — filter by recent activity and Python/JS stack
- Look for projects with good README documentation but missing evaluation logic — that is your opportunity
- ReportLab or WeasyPrint for generating structured PDF evaluation reports

TRACK E

Your Own Problem, Your Own Track

If you see a gap the above tracks do not cover — build for it

The four tracks above do not exhaust the design space. If your team has identified a specific, well-scoped problem within AI-powered interviewing that none of the above tracks address — a multi-modal interview agent that evaluates tone and pacing, an accessibility-first agent for neurodiverse candidates, a real-time coaching agent that helps candidates improve mid-session — you may define your own track. You must clearly articulate the problem, the solution, and why it matters in your 2-page writeup

Hints & Strategic Advice

The golden rule for a 9-hour hackathon

A small, deeply solved problem beats a large, shallowly attempted one. Judges would rather see one killer feature that works perfectly than five half-built features that do not.

Technical Choices

- Do not build an LLM from scratch. Use OpenAI, Groq (free Llama tier, very fast), or the Claude API for your agent backbone
- sentence-transformers (all-MiniLM-L6-v2) is fast, free, and runs locally. Use it for semantic similarity scoring.
- Streamlit can turn a Python backend into a demo in under 30 minutes. Do not spend 3 hours on a React frontend.
- Cache LLM responses aggressively during development. Rate limits will catch you off guard during live demos.
- For fraud detection, start with readability scores and embedding drift — both are fast to implement and visually demonstrable.

Explainability (Non-Negotiable)

Any AI system making hiring-adjacent decisions that cannot explain itself is a legal and ethical liability. At minimum, your agent must output which competencies were assessed, what evidence informed each score, and what the candidate's key gaps were. Judges will look for this.

- **At minimum:** A score per competency with one sentence of reasoning
- **Better:** A ranked summary — "Strongest signal: systems thinking. Weakest signal: communication clarity."
- **Best:** A candidate-facing version of the report with actionable, non-demoralising feedback on how to improve

On Avoiding Common Pitfalls

- Do not build a glorified FAQ chatbot and call it an AI interviewer. Judges will see through it immediately.
- Do not auto-reject candidates based on fraud signals alone — flag and explain. Automatic rejection is a design failure.
- Do not use real candidate data. Generate synthetic profiles and transcripts using an LLM.
- Do not hardcode the job description — let the judge paste one in during the live demo. It makes the system feel real

It is entirely up to your team which track you choose. Each track is equally weighted in judging. The only requirement is depth — pick one area and build something that genuinely works. One real, explainable, demonstrable feature beats five half-finished ideas every time.

Resources & Starting Points

DATASETS

- Resume Dataset — Kaggle (~2400 labelled resumes, 24 roles)
- LinkedIn Job Postings 2023 — Kaggle (real JDs with skills)
- Interview Question Datasets — GitHub / Kaggle
- Synthetic transcripts — generate with LLM

OPEN-SOURCE PROJECTS TO FORK

- GitHub search: "AI interviewer open source"
- GitHub search: "mock interview agent python"
- GitHub search: "interview chatbot LLM"
- Filter: Python, recent commits, permissive licence

SAMPLE LLM PROMPTS

- Adaptive follow-up: "Given this JD and the candidate's last answer, generate the most valuable follow-up question."
- Competency scoring: "Score this response 1-5 for [competency]. Return JSON: {score, evidence, gaps}"
- Fraud analysis: "Compare these two responses for vocabulary and structural consistency. Flag anomalies."

Evaluation & Judging

Criterion	Weight	What judges look for
Problem Depth & Relevance	25%	Does the system genuinely address one of the five impact areas, or is it a scripted chatbot with a new coat of paint?
Technical Execution	25%	Does the agent actually work? Is the questioning logic adaptive, or is it a static list? Is the code coherent and runnable?
Explainability & Fairness	20%	Can the system justify its evaluation outputs? Is there evidence of experience-level calibration and bias awareness?
Innovation	15%	Is there a genuinely novel approach — in domain specificity, fraud detection, or candidate experience — beyond a standard API wrapper?
Demo & Communication	15%	Can the team clearly explain what they built, why it matters, and what they would build next with more time?

What a winning submission looks like

A team picks Track C — Fraud Detection. They build an agent that conducts a 5-question interview, embeds each response, tracks semantic drift across the session, and flags two responses as anomalous with a confidence score and a one-line reason. A calibration toggle lets the demo audience switch between "fresher" and "senior" mode — the threshold shifts visibly. The UI is a single Streamlit page. It works. The demo is live. The code is readable. That team wins.

Deliverables

Round 1

- GitHub repo link
- 2-page technical writeup — which impact area you chose, your architecture in plain language, tools used, known limitations.
- Demo Video (2-3 mins) — talking us through the agent you have built with voiceover

Round 2 — Shortlisted teams

- 5-7 minute pitch deck — problem framing, solution walkthrough, live demo, and what you would build next !